ASSIGNMENT 3 – CO3

Constructors, Destructors and Operator Overloading in C++

Course: DSA0101 / DSA01 (edit to match course code)

Student Name: SK ALI BABU

Register Number: 192524038

Nature: Individual | Duration: 1 Week | Total Marks: 50

Submission Date: 01-11-2026

GitHub Repository: DSA01-192524038-Assignment3

GitHub Link: https://github.com/shaikalibabu143-alt/DSA0104-C-/upload/main/assessment%202

Q1 – Constructors and Constructor Overloading (6 Marks)

1. Design and implementation

A constructor initializes an object at the moment it is created. This Book class supports an empty record, a title-only donated book, and a fully specified book. The central constructor validates the copy count; delegating constructors reuse it.

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

class Book {
   string title;
   string author;
   int copies;

   static int checkedCopies(int n) {
      if (n < 0) throw invalid_argument("Copies cannot be negative");
      return n;
   }
public:
   Book() : Book("", "", 0) {}
   explicit Book(string t) : Book(t, "Unknown", 1) {}
   Book(string t, string a, int n)
      : title(t), author(a), copies(checkedCopies(n)) {}

   void print() const {
      cout << "Title: " << (title.empty() ? "(empty)" : title)
           << ", Author: " << author << ", Copies: " << copies << '\\n';
   }
};

int main() {
   Book empty;
   Book donated("C++ Basics");
   Book full("Clean Code", "Robert Martin", 4);
   empty.print(); donated.print(); full.print();
   try { Book invalid("Bad record", "Nobody", -2); invalid.print(); }
   catch (const invalid_argument& e) { cout << "Error: " << e.what() << '\\n'; }
}
```

2. Compiler and initialization rules

- When no constructor is declared, the compiler may implicitly declare a default constructor. Once a constructor is declared by the programmer, the compiler does not automatically add a no-argument default constructor.
- Member initializer lists initialize members directly and are required for const members and references. They avoid default construction followed by assignment and can be more efficient.
- Members initialize in the order they are declared in the class, not the order written in the initializer list. Write the list in declaration order to avoid confusion and -Wreorder warnings.
- Object states after construction: empty = (title="", author="", copies=0); donated = (title="C++ Basics", author="Unknown", copies=1); full = (title="Clean Code", author="Robert Martin", copies=4).
- If the default constructor is removed, `Book books[3];` fails because each array element needs a default constructor. `Book books[3] = {Book("A"), Book("B"), Book("C")};` can work if the other constructors allow it.
- Negative copies are rejected by throwing invalid_argument. Validation is centralized in checkedCopies(), so all constructors that delegate to the main constructor enforce the same rule.
- The title-only constructor is explicit to prevent implicit conversion from a string to Book. Without explicit, `Book b = string("Novel");` can compile; with explicit it fails. Direct initialization `Book b("Novel");` still compiles.

3. Demonstrating -Wreorder

```
// Compile separately with: g++ -std=c++17 -Wall -Wextra -Wreorder reorder.cpp
struct Example {
   int first;
   int second;
   Example() : second(2), first(1) {} // warning: list order differs from declaration order
};
```

4. Alternative: one constructor with defaults

```
class BookAlternative {
   std::string title, author;
   int copies;
public:
   BookAlternative(std::string t = "", std::string a = "Unknown", int n = 0)
     : title(std::move(t)), author(std::move(a)), copies(n) {
       if (copies < 0) throw std::invalid_argument("Copies cannot be negative");
   }
};
```

Both designs create a fixed-size object in O(1) time and O(1) object storage (not counting dynamic memory owned by strings). Several overloads make the intended situations clearer; default arguments reduce overload count but may make calls less self-documenting. With a single validating constructor, the check is written in one place. Delegation also lets the overloads share that one check.

5. Complexity

Construction: O(1) apart from string allocation/copying; object storage: O(1) excluding the string contents. Actual string work depends on title and author length.

Q2 – Copy Constructors: Shallow and Deep Copies

1. Complete Code (TODO 1–3)

```
#include <iostream>
#include <string>
#include <algorithm>
class Prescription {
   int n;
   std::string* medicines;
public:
   explicit Prescription(int count)
     : n(count), medicines(count > 0 ? new std::string[count] : nullptr) {}

   Prescription(const Prescription& other)
     : n(other.n), medicines(other.n > 0 ? new std::string[other.n] : nullptr) {
       for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
   }

   Prescription& operator=(const Prescription& other) {
      if (this != &other) {
         std::string* fresh = other.n > 0 ? new std::string[other.n] : nullptr;
         for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
         delete[] medicines;
         medicines = fresh;
         n = other.n;
      }
      return *this;
   }

   ~Prescription() { delete[] medicines; }
   void set(int i, const std::string& m) { if (i >= 0 && i < n) medicines[i] = m; }
   std::string get(int i) const { return (i >= 0 && i < n) ? medicines[i] : ""; }
};
int main() {
   Prescription p(2);
   p.set(0, "Paracetamol"); p.set(1, "ORS");
   Prescription q = p;
   q.set(1, "Cetirizine");
   Prescription r(1);
```

```
    r = p;
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | "
              << q.get(0) << ' ' << q.get(1) << " | "
              << r.get(0) << ' ' << r.get(1) << '\\n';
}
```

Expected output: `Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS`.

2. Explanation and memory diagrams

Shallow copy: p.medicines ──► [Paracetamol, ORS] ◄── q.medicines. Both objects own the same array. Changing one changes the other; when both destructors run, both call delete[] on the same pointer, causing double-free/undefined behaviour.

Deep copy: p.medicines ──► [Paracetamol, ORS] and q.medicines ──► [Paracetamol, ORS] are separate arrays. Changes and destruction are independent.

- Rule of Three: if a class needs a custom destructor, it commonly also needs a copy constructor and copy-assignment operator because it manages a resource.
- Copy constructor is used when initializing an object from another object, passing an object by value, and returning an object by value (copy elision may avoid the copy in modern C++).
- Pass-by-value may invoke a copy; pass-by-const-reference does not copy the Prescription object. A copy counter in the copy constructor can verify this.
- The compiler-generated copy constructor is appropriate when memberwise copying already has the intended value semantics, e.g. ints, strings, and vectors. It is unsafe for a raw owning pointer.

3. Sanitizer and shallow-copy experiment

Compile the deep-copy version with: `g++ -std=c++17 -Wall -Wextra -g -fsanitize=address -fno-omit-frame-pointer q2.cpp -o q2` and run `./q2` (Windows MinGW may not support AddressSanitizer; use a supported GCC/Clang environment or Valgrind). The deep-copy version should finish without a double-free report. In a separate intentionally shallow-copy version, the sanitizer is expected to report a double-free/use-after-free or related invalid memory operation. Paste your actual terminal log here.

4. Timing and complexity

For n medicine names, deep copy is O(n) time and O(n) additional space. A shallow pointer copy is O(1) time and O(1) additional space but is incorrect for independent ownership. Measure 10, 1,000 and 100,000 entries using std::chrono, repeat each measurement many times, and record your actual results in the table.

Medicine count	Deep-copy time (your measurement)	Sanitizer result
10	_____	_____
1,000	_____	_____
100,000	_____	_____

5. Rule of Zero alternative

```
#include <string>
#include <vector>
class PrescriptionVector {
   std::vector<std::string> medicines;
public:
   explicit PrescriptionVector(int n) : medicines(n > 0 ? n : 0) {}
   void set(std::size_t i, const std::string& m) { medicines.at(i) = m; }
   const std::string& get(std::size_t i) const { return medicines.at(i); }
};
```

std::vector owns its storage and performs a deep value copy, so custom copy/destructor functions are unnecessary (Rule of Zero). Copying n medicine names is O(n) time and O(n) extra space.

Q3 – Dynamic Constructors and Destructors (7 Marks)

1. Dynamic seat map implementation

```
#include <iostream>
#include <stdexcept>
#include <string>
class Event {
   int rows, seatsPerRow;
   bool** seats;
   std::string name;
public:
   Event(std::string eventName, int r, int s)
     : rows(r), seatsPerRow(s), seats(nullptr), name(std::move(eventName)) {
       if (r < 0 || s < 0) throw std::invalid_argument("Negative dimensions");
       if (rows == 0) { std::cout << "Created " << name << " with zero rows\\n"; return; }
       seats = new bool*[rows]{};
       try {
          for (int i=0; i<rows; ++i) seats[i] = new bool[seatsPerRow]{};
       } catch (...) {
          for (int i=0; i<rows; ++i) delete[] seats[i];
          delete[] seats;
          throw;
       }
       std::cout << "Created " << name << " (" << rows << " rows)\\n";
   }
   Event(const Event&) = delete;
   Event& operator=(const Event&) = delete;
   bool book(int r, int s) {
       if (!seats || r<0 || r>=rows || s<0 || s>=seatsPerRow || seats[r][s]) return false;
       seats[r][s] = true; return true;
```

```
    }
    bool cancel(int r, int s) {
        if (!seats || r<0 || r>=rows || s<0 || s>=seatsPerRow || !seats[r][s]) return false;
        seats[r][s] = false; return true;
    }
    ~Event() {
        for (int i=0; i<rows; ++i) delete[] seats[i];
        delete[] seats;
        std::cout << "Closed " << name << '\\n';
    }
};
int main() {
    Event local("Local", 2, 3);
    std::cout << "Book result: " << local.book(0,1) << '\\n';
    { Event block("Block", 1, 2); block.book(0,0); }
    Event* heap = new Event("Heap", 3, 4);
    heap->book(1,1); delete heap;
    Event zero("Zero", 0, 5);
}
```

2. Lifecycle and memory

- Local automatic objects are destroyed in reverse order of construction when their scope ends. Block-scope objects are destroyed at the closing brace. Heap objects created by new remain alive until delete is called.
- If delete is forgotten, the object and its allocations leak. For arrays allocated with new[], use delete[]; using scalar delete on an array is undefined behaviour.
- A destructor has no parameters and no return type, and it cannot be overloaded because destruction has one fixed operation for an object. It can be virtual in a polymorphic base class.
- Seat-map layout: seats ⟶ row-pointer array [ptr0, ptr1, ...]; each ptr points to a separately allocated bool array for that row.

3. Allocation measurements

Assuming one bool occupies one byte, the seat data alone uses approximately rows × seats bytes: 10×20 = 200 bytes; 100×200 = 20,000 bytes; 1,000×2,000 = 2,000,000 bytes (about 1.91 MiB). This excludes row pointers, allocator metadata and object overhead. Record measured process/heap figures if available.

Seat map	Seat-data estimate	Measured total (optional)
10 × 20	200 bytes	_____
100 × 200	20,000 bytes	_____
1,000 × 2,000	2,000,000 bytes	_____

4. Predict the Logger output

Write your own prediction before executing. Expected ordering under ordinary C++17 execution, with copy elision not applying to the by-value parameter from the lvalue `a`, is:

+A +B -B copyAc use -Ac +P -P end -A
Explanation: constructing a and b prints +A and +B; b is destroyed at its block end. Passing a by value creates a copied parameter named Ac, prints copyAc, then use prints use; the parameter is destroyed after use returns, printing -Ac. P is dynamically created and prints +P, then delete prints -P. `end` is printed before a is destroyed at main's end, so -A is last.

5. Alternative storage designs
A single contiguous allocation of rows×seats elements uses one allocation and index `r * seatsPerRow + s`; access remains O(1). A vector<vector<char>> is safer and automatically managed but commonly uses one allocation for the outer vector plus one per row. The row-pointer design uses rows+1 allocations and rows pointers (O(rows) pointer overhead). RAII ties release to object lifetime, including early returns and exceptions, reducing leaks.

Sanitizer task: make a separate intentionally faulty version that omits delete for the heap Event, run under AddressSanitizer/LeakSanitizer, save the actual report, then run the correct version. Do not fabricate sanitizer output.

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

1. Complete Code (TODO 1–2)
```
#include <iomanip>
#include <iostream>
class Money {
   long long paise;
public:
   Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
   Money operator+(const Money& other) const { Money result; result.paise = paise + other.paise;
return result; }
   bool operator==(const Money& other) const { return paise == other.paise; }
   friend std::ostream& operator<<(std::ostream& out, const Money& m) {
      long long whole = m.paise / 100, rem = m.paise % 100;
      if (rem < 0) rem = -rem;
      if (m.paise < 0 && whole == 0) out << "Rs -0." << std::setw(2) << std::setfill('0') << rem;
      else out << "Rs " << whole << '.' << std::setw(2) << std::setfill('0') << rem;
      out << std::setfill(' ');
      return out;
   }
};
int main() {
   Money a(120,75), b(79,25);
   Money c = a + b;
   std::cout << c << '\\n' << std::boolalpha << (c == Money(200)) << '\\n';
   Money x(0,75), y(0,50);
```

```
    std::cout << x + y << '\\n';
}
```
Expected output: `Rs 200.00`, `true`, then `Rs 1.25` on separate lines.

2. Operator design

- `a + b` is interpreted as `a.operator+(b)` for a member overload. + normally returns a new value; += mutates the left operand and returns `Money&` so chained assignments work without an extra copy.
- == can be a member because it compares this object with the right operand. << is a non-member (often friend) because the left operand is an ostream, not Money. A friend grants access to private data but should be used only when needed.
- To support +=, add `Money& operator+=(const Money& rhs) { paise += rhs.paise; return *this; }`. Subtraction should define whether negative balances are allowed; this version permits negative values.
- Multiplying Money by Money is usually dimensionally confusing (currency squared). Multiplying/dividing Money by a scalar may be meaningful if rounding rules are specified.

3. Reading operator and alternative

```
friend std::istream& operator>>(std::istream& in, Money& m) {
    long long rupees; int paisePart;
    if (in >> rupees >> paisePart) m.paise = rupees * 100 + paisePart;
    return in;
}
```
Named functions add(a,b) and equals(a,b) have the same O(1) time and O(1) storage as operators. Non-member operators can support implicit conversions on either operand; for example `100 + m` is possible with a non-member overload accepting a Money-convertible left operand, while a member overload is called only on an existing Money left operand. Be cautious with implicit conversions and prefer explicit construction if ambiguity is possible.

For a double-based implementation, binary floating-point rounding can accumulate over many additions. Integer paise avoids fractional representation errors for whole paise values. To compare one million additions, run repeated timed tests and report your actual timing.

Implementation	1,000,000 additions time
Integer paise	_____
double rupees	_____

Q5 – Increment, Comparison and Subscript Operators (6 Marks)

1. Counter prediction

Predicted output: `5 7 7`.

Initially c.v is 5. `a = c++` copies the old value 5 into a, then increments c to 6. `b = ++c` increments c to 7 first and returns the updated object by reference, so b becomes 7. Final c is 7. Postfix makes a copy; prefix does not require a copy.

2. Time and Playlist reference implementation

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
class Time {
   int seconds;
public:
   explicit Time(int s=0) : seconds((s % 86400 + 86400) % 86400) {}
   Time& operator++() { seconds = (seconds + 1) % 86400; return *this; }
   Time operator++(int) { Time old=*this; ++(*this); return old; }
   bool operator==(const Time& x) const { return seconds == x.seconds; }
   bool operator<(const Time& x) const { return seconds < x.seconds; }
   friend std::ostream& operator<<(std::ostream& o, const Time& t) {
      return o << t.seconds/3600 << ':' << (t.seconds/60)%60 << ':' << t.seconds%60;
   }
};
class Playlist {
   std::vector<std::string> tracks;
public:
   explicit Playlist(std::vector<std::string> t) : tracks(std::move(t)) {}
   std::string& operator[](int i) {
      if (i<0 || i>=static_cast<int>(tracks.size())) throw std::out_of_range("track index");
      return tracks[i];
   }
   const std::string& operator[](int i) const {
      if (i<0 || i>=static_cast<int>(tracks.size())) throw std::out_of_range("track index");
      return tracks[i];
   }
};
int main() {
   Time t(59*60+59);
   Time a=t++; Time b=++t;
   std::cout << a << " | " << b << " | " << t << '\\n';
   Playlist p({"Track A","Track B"});
   std::cout << p[0] << '\\n';
   try { std::cout << p[-1]; } catch (const std::out_of_range& e) { std::cout << "Error: " << e.what() <<
'\\n'; }
}
```

The Time implementation wraps at 24 hours; it displays 1:0:0 rather than zero-padded 1:00:00. Formatting can be improved with setw/setfill. The same seconds representation correctly rolls 59:59 into 1:0:0.

3. Rules and complexity

- Prefix is `operator++()`; postfix is `operator++(int)`, where the unused int distinguishes the signature.
- Postfix must return the old value, so it typically copies the object. Prefix returns a reference to the incremented object.
- Non-const operator[] returns a modifiable reference; const operator[] returns a const reference. A range-checking version can throw out_of_range, return a sentinel, or print an error; throwing is generally easier to handle correctly than printing inside the class.
- Increment and indexed access are O(1). Checked access performs an extra comparison. Benchmark 10,000,000 increments under `-O0` and `-O2`; ensure the compiler cannot optimize away the loop, and record real measurements.

Benchmark	Time (your measurement)
Prefix ++, -O0, 10,000,000	_____
Postfix ++, -O0, 10,000,000	_____
Prefix ++, -O2, 10,000,000	_____
Postfix ++, -O2, 10,000,000	_____
Checked access, 10,000,000	_____
Unchecked access, 10,000,000	_____

Programmers often prefer ++i for class types because it avoids producing an old-value copy when the old value is not needed.

Q6 – The Rules of Operator Overloading (7 Marks)

1. Safe Text class

```
#include <cstring>
#include <iostream>
#include <stdexcept>
class Text {
   char* data;
public:
   Text(const char* s = "") {
      if (!s) s = "";
      data = new char[std::strlen(s)+1];
      std::strcpy(data, s);
   }
   Text(const Text& other) : Text(other.data) {}
   Text& operator=(const Text& other) {
      if (this != &other) {
         char* fresh = new char[std::strlen(other.data)+1];
         std::strcpy(fresh, other.data);
         delete[] data;
         data = fresh;
      }
```

```
        return *this;
    }
    ~Text() { delete[] data; }
    Text& operator+=(const Text& rhs) {
        std::size_t a=std::strlen(data), b=std::strlen(rhs.data);
        char* fresh = new char[a+b+1];
        std::memcpy(fresh, data, a);
        std::memcpy(fresh+a, rhs.data, b+1);
        delete[] data;
        data=fresh;
        return *this;
    }
    friend Text operator+(const Text& a, const Text& b) {
        Text result(a); result += b; return result;
    }
    bool operator==(const Text& rhs) const { return std::strcmp(data,rhs.data)==0; }
    bool operator!=(const Text& rhs) const { return !(*this==rhs); }
    friend std::ostream& operator<<(std::ostream& out, const Text& t) { return out << t.data; }
};
int main() {
    Text a("Hello "), b("world"), c("!");
    Text result = a + b + c;
    std::cout << result << '\\n';
    Text empty;
    std::cout << "empty=[" << empty << "]\\n";
    a = a; // self-assignment is safe
    std::cout << std::boolalpha << (a != b) << '\\n';
}
```

2. Language rules

- Operators that cannot be overloaded include ::, ., .*, ?: and sizeof.
- An overloaded operator cannot change precedence, associativity, or number of operands (arity). At least one operand must be a user-defined type; operators cannot be redefined for two built-in types only.
- operator= must be a non-static member function. A friend/non-member function is useful for symmetric operations and stream insertion, where the left operand is not the class.
- Rule of Three applies because Text owns a raw dynamically allocated character array. The assignment operator builds new storage before deleting old storage, making self-assignment safe and avoiding loss of the original data if allocation fails.

3. Copy count, sanitizer and performance

For `a + b + c`, the expression is left-associative: `(a+b)+c`. In this implementation, each + constructs a result copy of its left operand, then += appends. Exact copy-constructor counts can depend on copy elision; add a counter to the copy constructor and record the actual compiler/version/flags.

AddressSanitizer should show no invalid access for this implementation. Compare 100,000 concatenations against std::string using the same input sizes and build flags; append into a growing string/vector or reserve capacity where appropriate.

For lengths n1, n2, n3, computing a+b+c copies roughly n1+n2 characters for the first concatenation and then (n1+n2)+n3 for the second, so total character work is O(n1+n2+n3), though repeated left-associated concatenations in a longer chain can cause repeated copying and quadratic total work. Temporary buffers require O(n1+n2+n3) peak space for a single expression.

4. std::string alternative

std::string already manages allocation, copying, assignment, concatenation, comparisons and output. It is the preferred production choice in ordinary C++ code. Writing Text demonstrates ownership, deep copying, exception safety and operator design; a custom class is justified when a special representation, protocol, embedded constraint or learning objective requires it.

Benchmark	Time (your measurement)
Text, 100,000 concatenations	_____
std::string, 100,000 concatenations	_____

Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Stage 1 – Interpret the problem (2 marks)

Data: real component and imaginary component, both double values. Required operations: addition, subtraction, multiplication, division, equality, output, magnitude. Constructors: default (0+0j), real-only (x+0j), and real/imaginary (x+yj). Requirements: arithmetic results match formulas; division by zero is handled safely; output shows rectangular form and magnitude.

Stage 2 – Design and formulas (3 marks)

- Addition: (a+bi)+(c+di) = (a+c)+(b+d)i.
- Subtraction: (a+bi)-(c+di) = (a-c)+(b-d)i.
- Multiplication: (a+bi)(c+di) = (ac-bd)+(ad+bc)i.
- Division: (a+bi)/(c+di) = [(ac+bd)+(bc-ad)i]/(c^2+d^2), provided c^2+d^2 is nonzero.
- Magnitude: |z| = sqrt(a^2+b^2). Equality compares both components; exact floating-point equality may be unsuitable for numerical calculations.

Stage 3 – Implementation and verification (3 marks)

```
#include <cmath>
#include <iostream>
#include <stdexcept>
class Complex {
   double re, im;
public:
   Complex() : re(0), im(0) {}
   Complex(double real) : re(real), im(0) {}
   Complex(double real, double imag) : re(real), im(imag) {}
```

```
    double real() const { return re; }
    double imag() const { return im; }
    double magnitude() const { return std::sqrt(re*re + im*im); }
    Complex operator+(const Complex& z) const { return {re+z.re, im+z.im}; }
    Complex operator-(const Complex& z) const { return {re-z.re, im-z.im}; }
    Complex operator*(const Complex& z) const {
        return {re*z.re-im*z.im, re*z.im+im*z.re};
    }
    Complex operator/(const Complex& z) const {
        double den = z.re*z.re + z.im*z.im;
        if (den == 0.0) throw std::domain_error("division by zero impedance");
        return {(re*z.re+im*z.im)/den, (im*z.re-re*z.im)/den};
    }
    bool operator==(const Complex& z) const { return re==z.re && im==z.im; }
    friend std::ostream& operator<<(std::ostream& out, const Complex& z) {
        return out << z.re << (z.im < 0 ? " - " : " + ")
                   << std::abs(z.im) << "j";
    }
};
int main() {
    const double R=10.0, w=100.0, L=0.1, C=0.001;
    Complex zr(R,0), zl(0,w*L), zc(0,-1.0/(w*C));
    Complex series = zr + zl + zc;
    std::cout << "Series impedance: " << series
              << ", magnitude=" << series.magnitude() << '\\n';
    Complex z1(10,5), z2(20,-2);
    Complex parallel = (z1*z2)/(z1+z2);
    std::cout << "Parallel impedance: " << parallel
              << ", magnitude=" << parallel.magnitude() << '\\n';
    try {
        Complex bad = z1 / Complex(0,0);
        std::cout << bad << '\\n';
    } catch (const std::domain_error& e) {
        std::cout << "Error: " << e.what() << '\\n';
    }
}
```

Hand check for the series example: R=10 Ω, ω=100 rad/s, L=0.1 H gives XL=ωL=10 Ω. C=0.001 F gives XC=1/(ωC)=10 Ω. Therefore Z=10+j10−j10=10+j0 Ω and magnitude is 10 Ω.

The parallel example uses Z1=10+5j and Z2=20−2j. Calculate the product and sum using the formulas, then divide the product by the sum; compare those hand-calculated values with the program output. Division by 0+0j must print the handled error rather than crash.

Stage 4 – Defend the design (2 marks)

- Constructors provide natural ways to represent zero impedance, a real-only resistance and a full complex impedance.
- Arithmetic operators keep circuit formulas readable. << is a friend/non-member because ostream is the left operand; arithmetic operators can be members.
- std::complex<double> provides tested complex arithmetic, magnitude and stream support. Both representations store two doubles and use O(1) time per basic operation and O(1) storage. A custom class requires more testing, while std::complex has standard-library support and well-tested numerical behaviour.
- Limitation: exact equality on double values can be unreliable after floating-point calculations; use a tolerance such as abs(a-b) < epsilon when appropriate. The simple division guard may also need a tolerance for near-zero denominators.

Alternative concept and complexity

With std::complex<double>, use `std::complex<double> z1(a,b), z2(c,d); auto sum=z1+z2; auto quotient=z1/z2;`. Basic operations remain O(1) time and O(1) space (two doubles per value). Division by zero should still be checked by the application because floating-point complex division may produce infinities or NaNs rather than throw an exception.

Final Submission Checklist

- Create repository `DSA01-192524038-Assignment3` and share it with faculty.
- Create folders Task01 through Task07; put source files and output screenshots in each folder.
- Compile each program with a documented command, e.g. `g++ -std=c++17 -Wall -Wextra -Wreorder -pedantic q1.cpp -o q1`.
- Use separate source files for intentionally faulty/shallow-copy/missing-delete demonstrations; keep the final correct versions too.
- Record actual outputs, predictions made before running, sanitizer logs and benchmark timings. Do not invent experimental results.
- Add README.md with student name, register number, compiler version, compile/run commands, and a short result note for each task.